



Security Audit

Report for Launchpad

Contracts

Date: October 8, 2026 **Version:** 1.0

Contact: contact@blocksec.com

Contents

Chapter 1 Introduction	1
1.1 About the Audit Target	1
1.2 Disclaimer	1
1.3 Procedure of Auditing	2
1.3.1 Security Issues	2
1.3.2 Additional Recommendation	3
1.4 Security Model	3
Chapter 2 Findings	4
2.1 Security Issue	5
2.1.1 Lack of slippage protection in the function <code>processExternalTax()</code>	5
2.1.2 Potential loss of funds due to the instant collateral withdrawal	6
2.1.3 Potential ineffective slippage checks in the function <code>buyExactTokens()</code>	8
2.1.4 Potential circumvention of liquidations	10
2.1.5 Potential DoS due to the malicious creation of positions	11
2.1.6 Potential sandwich attack targeting liquidations	12
2.1.7 Lack of minimum check for the variable <code>notionalNative</code> in the function <code>partialCloseShort()</code>	13
2.2 Recommendation	13
2.2.1 Revise the approval logic in the function <code>processExternalTax()</code>	13
2.2.2 Unify the native transfer method	14
2.2.3 Add a recipient check in the function <code>claimMigrationDust()</code>	15
2.2.4 Remove redundant code	15
2.2.5 Add a sweep mechanism in the contract <code>BigShortTaxProcessorV1</code>	16
2.2.6 Unify the reserved storage gap	17
2.2.7 Revise the margin-related checks in the contract <code>EconomicValidation</code>	17
2.3 Note	18
2.3.1 Potential centralization risks	18
2.3.2 Ensure correct market grouping	18
2.3.3 Ensure proper adaptations for non-Arc chain deployments	18

Report Manifest

Item	Description
Client	Bigshort
Target	Launchpad Contracts

Version History

Version	Date	Description
1.0	October 8, 2026	First release

Signature



Digitally signed by: BlockSec (<https://blocksec.com>)

SHA1 digest of public key: BAB106D3708BC6DE0963CE05E9478C24B326AE1C

Date: 2026-10-09 09:01:03+0000

About BlockSec BlockSec focuses on the security of the blockchain ecosystem and collaborates with leading DeFi projects to secure their products. BlockSec is founded by top-notch security researchers and experienced experts from both academia and industry. They have published multiple blockchain security papers in prestigious conferences, reported several zero-day attacks of DeFi applications, and successfully protected digital assets that are worth more than 14 million dollars by blocking multiple attacks. They can be reached at [Email](#), [Twitter](#) and [Medium](#).

Chapter 1 Introduction

1.1 About the Audit Target

Information	Description
Type	Smart Contract
Language	Solidity
Approach	Semi-automatic and manual verification

The audit target (hereinafter referred to as the Target) of this audit is the code repository ¹ of Launchpad Contracts of Bigshort.

Launchpad Contracts of BigShort is a token launchpad protocol that allows users to create meme tokens. Each token is first traded on an internal bonding curve and migrated to an external pool once the expected amount is reached. After migration, users can open short positions on any tokens through the Engine contract. Specifically, users borrow tokens from the Engine contract and sell them into the pool. Moreover, a TWAP oracle is used for calculating the short capacity, and unhealthy positions can only be liquidated by whitelisted liquidators.

Note this audit only focuses on the codes located in the following directory/file.

- src

Meanwhile, the following directory/file in the aforementioned directory is excluded.

- src/interfaces

Other files are not within the scope of the audit. Additionally, all dependencies of the Target are considered reliable in terms of both functionality and security, and are therefore not included in the audit scope.

The auditing process is iterative. Specifically, we would audit the commits that fix the discovered issues. If there are new issues, we will continue this process. The commit SHA values during the audit are shown in the following table. Our audit report is responsible for the code in the initial version ([Version 1](#)), as well as new code (in the following versions) to fix issues in the audit report. Code prior to and including the baseline version ([Version 0](#)), where applicable, is outside the scope of this audit and assumes to be reliable and secure.

Project	Version	Commit Hash
BigshortProtocol-EVM	Version 1	039cd8c34b934594d76dbba95d0f2fdb9e793a48
	Version 2	b71bd1ddbef41e997921a818dbab52112af1d864
	Version 3	2742c2af46d92a51c3c032b5b527e5b1efc5724a

1.2 Disclaimer

This audit report does not constitute investment advice or a personal recommendation. It does not consider, and should not be interpreted as considering or having any bearing on, the potential economics of a token, token sale or any other product, service or other asset.

¹<https://github.com/BigShortLTD/BigshortProtocol-EVM>

Any entity should not rely on this report in any way, including for the purpose of making any decisions to buy or sell any token, product, service or other asset.

This audit report is not an endorsement of any particular project or team, and the report does not guarantee the security of any particular project. This audit does not give any warranties on discovering all security issues of the Target, i.e., the evaluation result does not guarantee the nonexistence of any further findings of security issues. As one audit cannot be considered comprehensive, we always recommend proceeding with independent audits and a public bug bounty program to ensure the security of the Target.

The scope of this audit is limited to the code mentioned in Section 1.1. Unless explicitly specified, the security of the language itself (e.g., the solidity language), the underlying compiling toolchain and the computing infrastructure are out of the scope.

1.3 Procedure of Auditing

We perform the audit according to the following procedure.

- **Vulnerability Detection** We first scan the Target with automatic code analyzers, and then manually verify (reject or confirm) the issues reported by them.
- **Semantic Analysis** We study the business logic of the Target and conduct further investigation on the possible vulnerabilities using an automatic fuzzing tool (developed by our research team). We also manually analyze possible attack scenarios with independent auditors to cross-check the result.
- **Recommendation** We provide some useful advice to developers from the perspective of good programming practice, including gas optimization, code style, and etc.

We show the main concrete checkpoints in the following.

1.3.1 Security Issues

- * Access control
- * Permission management
- * Whitelist and blacklist mechanisms
- * Initialization consistency
- * Improper use of the proxy system
- * Reentrancy
- * Denial of Service (DoS)
- * Untrusted external call and control flow
- * Exception handling
- * Data handling and flow
- * Events operation
- * Error-prone randomness
- * Oracle security
- * Business logic correctness
- * Semantic and functional consistency
- * Emergency mechanism

- * Economic and incentive impact

1.3.2 Additional Recommendation

- * Gas optimization
- * Code quality and style



Note The previous checkpoints are the main ones. We may use more checkpoints during the auditing process according to the functionality of the project.

1.4 Security Model

To evaluate the risk, we follow the standards or suggestions that are widely adopted by both industry and academy, including OWASP Risk Rating Methodology² and Common Weakness Enumeration³. The overall *severity* of the risk is determined by *likelihood* and *impact*. Specifically, likelihood is used to estimate how likely a particular vulnerability can be uncovered and exploited by an attacker, while impact is used to measure the consequences of a successful exploit.

In this report, both likelihood and impact are categorized into two ratings, i.e., *high* and *low* respectively, and their combinations are shown in Table 1.1.

Table 1.1: Vulnerability Severity Classification

Impact	High	High	Medium
	Low	Medium	Low
		High	Low
		Likelihood	

Accordingly, the severity measured in this report are classified into three categories: **High**, **Medium**, **Low**. For the sake of completeness, **Undetermined** is also used to cover circumstances when the risk cannot be well determined.

Furthermore, the status of a discovered item will fall into one of the following five categories:

- **Undetermined** No response yet.
- **Acknowledged** The item has been received by the client, but not confirmed yet.
- **Confirmed** The item has been recognized by the client, but not fixed yet.
- **Partially Fixed** The item has been confirmed and partially fixed by the client.
- **Fixed** The item has been confirmed and fixed by the client.

²https://owasp.org/www-community/OWASP_Risk_Rating_Methodology

³<https://cwe.mitre.org/>

Chapter 2 Findings

In total, we found **seven** potential security issues. Besides, we have **seven** recommendations and **three** notes.

- High Risk: 2
- Medium Risk: 5
- Recommendation: 7
- Note: 3

ID	Severity	Description	Category	Status
1	High	Lack of slippage protection in the function <code>processExternalTax()</code>	Security Issue	Fixed
2	High	Potential loss of funds due to the instant collateral withdrawal	Security Issue	Fixed
3	Medium	Potential ineffective slippage checks in the function <code>buyExactTokens()</code>	Security Issue	Fixed
4	Medium	Potential circumvention of liquidations	Security Issue	Fixed
5	Medium	Potential DoS due to the malicious creation of positions	Security Issue	Fixed
6	Medium	Potential sandwich attack targeting liquidations	Security Issue	Fixed
7	Medium	Lack of minimum check for the variable <code>notionalNative</code> in the function <code>partialCloseShort()</code>	Security Issue	Confirmed
8	-	Revise the approval logic in the function <code>processExternalTax()</code>	Recommendation	Fixed
9	-	Unify the native transfer method	Recommendation	Fixed
10	-	Add a recipient check in the function <code>claimMigrationDust()</code>	Recommendation	Fixed
11	-	Remove redundant code	Recommendation	Fixed
12	-	Add a sweep mechanism in the contract <code>BigShortTaxProcessorV1</code>	Recommendation	Fixed
13	-	Unify the reserved storage gap	Recommendation	Fixed
14	-	Revise the margin-related checks in the contract <code>EconomicValidation</code>	Recommendation	Confirmed
15	-	Potential centralization risks	Note	-
16	-	Ensure correct market grouping	Note	-
17	-	Ensure proper adaptations for non-Arc chain deployments	Note	-

The details are provided in the following sections.

2.1 Security Issue

2.1.1 Lack of slippage protection in the function processExternalTax()

Severity High

Status Fixed in [Version 3](#)

Introduced by [Version 1](#)

Description In the contract [BigShortTaxProcessorV1](#), the function [processExternalTax\(\)](#) swaps the accumulated taxes and distributes them to beneficiaries. Specifically, once the balance of the contract [BigShortTaxProcessorV1](#) reaches the tax processing threshold (i.e., the variable [taxSwapThreshold](#)), the function [processExternalTax\(\)](#) will be invoked to swap the accumulated base token for the quote token without the slippage protection (i.e., the input [amountOutMin](#) is set to 0). However, this design is vulnerable allowing an attacker to gain profits by performing the following actions:

1. Open a short position, which sells the base tokens for the quote token and drives down the token price in the pair.
2. Trigger the tax process by transferring the base token to the pair. This step further drives down the price in the pair due to the lack of slippage protection.
3. Close the short position to realize a profit.

As a result, the attacker can gain profits by leveraging the vulnerable tax process.

```

191     if (to == pair) {
192         // Process only before the current sell reaches the Pair. Processing after this
            transfer
193         // would let the processor's swap consume the user's unswapped Pair input.
194         uint256 baseAmount = balanceOf(address(processor));
195         uint256 threshold = taxSwapThreshold;
196         if (baseAmount >= threshold) {
197             _processingExternalTax = true;
198             try processor.processExternalTax() returns (
199                 int8 direction, uint256 actualQuoteOutput, uint256 expectedQuoteOutput
200             ) {
201                 _processingExternalTax = false;
202                 _adjustTaxSwapThreshold(direction, actualQuoteOutput, expectedQuoteOutput);
203             } catch (bytes memory reason) {
204                 _processingExternalTax = false;
205                 emit ExternalTaxProcessingFailed(address(processor), from, baseAmount, threshold
                    , reason);
206             }
207         }

```

Listing 2.1: src/v1/BigShortTokenV1.sol

```

254     try router.swapExactTokensForTokensSupportingFeeOnTransferTokens(
255         baseAmount, 0, path, address(this), block.timestamp

```

Listing 2.2: src/v1/BigShortTaxProcessorV1.sol

Impact The attacker can gain profits by leveraging the vulnerable tax process.

Suggestion Revise the logic accordingly.

2.1.2 Potential loss of funds due to the instant collateral withdrawal

Severity High

Status Fixed in [Version 3](#)

Introduced by [Version 1](#)

Description In the contract [BigShortEngineV1](#), the function [openShort\(\)](#) allows users to open a short position with sufficient collateral (i.e., the quote token). This function later swaps the borrowed token (i.e., the base token) for the quote token, which is added to the user's margin (i.e., the variable [marginNative](#)). However, the function [withdrawCollateral\(\)](#) allows users to instantly withdraw collateral as long as their positions are healthy. This design allows a malicious users to gain profits with the following actions:

1. Open a position A, which decreases the base token price (i.e., P1) in the external pool.
2. Open a position B, which further decreases the base token price (i.e., P2).
3. Withdraw the margin from the position B via the function [withdrawCollateral\(\)](#).
4. Close position A with a lower price P2 to gain profits.

As a result, the malicious user can gain profits by leveraging the instant withdrawal (i.e., via the function [withdrawCollateral\(\)](#)).

```

720 function withdrawCollateral(
721     uint256 positionId,
722     uint256 amount,
723     address payable recipient
724 ) external ready nonReentrant {
725     if (amount == 0 || recipient == address(0)) revert InvalidAmount();
726     Position storage position = positions[positionId];
727     _requireOwner(positionId, position);
728     _updateFunding();
729     (, , uint256 unpaid) = _settleAllFunding(positionId, position);
730     if (unpaid != 0) revert FundingExceedsMargin();
731     (uint112 tokenReserve, uint112 quoteReserve) = _validatedPairReserves();
732     (bool feasible, uint256 debtValue) = _nativeRequiredForExactTokens(
733         position.borrowedTokens,
734         quoteReserve,
735         tokenReserve
736     );
737     if (!feasible) revert ShortCloseNotAllowed();
738     uint256 requiredMargin = Math.mulDiv(
739         debtValue,
740         C.SHORT_SAFE_MARGIN_BPS,
741         C.BPS,
742         Math.Rounding.Ceil
743     );
744     uint256 excess = position.marginNative > requiredMargin
745         ? position.marginNative - requiredMargin
746         : 0;
747     uint256 maxWithdrawable = Math.min(excess, position.collateralNative);
748     if (amount > maxWithdrawable) revert InsufficientCollateral();
749     position.collateralNative -= amount;
750     position.marginNative -= amount;

```

```
751     totalPositionMarginNative -= amount;
752     _sendNative(recipient, amount);
753     emit CollateralWithdrawn(positionId, recipient, amount);
754     _emitPositionStateUpdated(positionId);
755     _emitEngineStateUpdated();
756 }
```

Listing 2.3: src/v1/BigShortEngineV1.sol

```
860 function closeShort(
861     uint256 positionId,
862     uint256 maxNativeIn,
863     address payable payoutRecipient
864 )
865     external
866     ready
867     nonReentrant
868     returns (uint256 buybackNative, uint256 payoutNative)
869 {
870     if (maxNativeIn == 0 || payoutRecipient == address(0))
871         revert InvalidAmount();
872     Position storage position = positions[positionId];
873     _requireOwner(positionId, position);
874     _updateFunding();
875     (
876         uint256 fundingOwed,
877         uint256 fundingPaid,
878         uint256 fundingUnpaid
879     ) = _settleAllFunding(positionId, position);
880     if (fundingUnpaid != 0) revert FundingExceedsMargin();
881     RiskState memory risk = riskState();
882     (uint112 tokenReserve, uint112 quoteReserve) = _validatedPairReserves();
883     (bool feasible, uint256 required) = _nativeRequiredForExactTokens(
884         position.borrowedTokens,
885         quoteReserve,
886         tokenReserve
887     );
888     uint256 safeBudget = Math.mulDiv(
889         position.marginNative,
890         C.BPS,
891         risk.marginRatioBps
892     );
893     if (
894         !feasible ||
895         required > position.marginNative ||
896         required > safeBudget ||
897         required > maxNativeIn
898     ) {
899         revert ShortCloseNotAllowed();
900     }
901     buybackNative = _buyExactTokens(position.borrowedTokens, required);
902     payoutNative = position.marginNative - buybackNative;
903     address positionOwner = position.owner;
```

```
904     uint256 legacyQueued = _closePosition(  
905         positionId,  
906         position,  
907         position.borrowedTokens,  
908         true  
909     );  
910     emit ShortClosed(  
911         positionId,  
912         positionOwner,  
913         payoutRecipient,  
914         fundingOwed,  
915         fundingPaid,  
916         fundingUnpaid,  
917         buybackNative,  
918         payoutNative,  
919         legacyQueued  
920     );  
921     if (payoutNative != 0) _sendNative(payoutRecipient, payoutNative);  
922     _emitPositionStateUpdated(positionId);  
923     _emitEngineStateUpdated();  
924 }
```

Listing 2.4: src/v1/BigShortEngineV1.sol

```
494     function availableLending() public view returns (uint256) {  
495         return  
496             initialLendingTokens -  
497             unrecoveredLendingTokens -  
498             totalBorrowedTokens;  
499     }
```

Listing 2.5: src/v1/BigShortEngineV1.sol

Impact The malicious user can gain profits by leveraging the instant withdrawal (i.e., via the function `withdrawCollateral()`).

Suggestion Revise the code accordingly.

2.1.3 Potential ineffective slippage checks in the function `buyExactTokens()`

Severity Medium

Status Fixed in [Version 3](#)

Introduced by [Version 1](#)

Description In the contract `BigShortFactoryV1`, the function `buyExactTokens()` allows users to buy base tokens by specifying the maximum cost (i.e., the input `maxTotalCost`) as the slippage protection. However, this function clamps the output to `min(requestedTokenAmount, curve.realTokenReserves)` during the migration and validates `maxTotalCost` only against the clamped cost. As a result, this design may force users to trade at an unexpected price during the migration.

```
503     function buyExactTokens(  
504         address token,
```

```
505     uint256 requestedTokenAmount,
506     uint256 maxTotalCost,
507     uint256 deadline,
508     address recipient
509 ) external payable nonReentrant whenNotPaused returns (uint256 tokenAmount, uint256 totalCost)
    {
510     _requireFactorySolventBeforePayableCall();
511     _checkDeadline(deadline);
512     Curve storage curve = _bondingCurve(token);
513     BigShortCurveConfig memory curveConfig = _curveConfigForToken(token);
514     address livePair = curve.pair;
515     if (requestedTokenAmount == 0 || recipient == address(0) || recipient == livePair) {
516         revert InvalidAmount();
517     }
518     tokenAmount = Math.min(requestedTokenAmount, curve.realTokenReserves);
519     uint256 curveNet =
520         BigShortMath.buyQuote(curve.virtualNativeReserves, curve.virtualTokenReserves,
521                               tokenAmount);
522     uint256 protocolFee = BigShortMath.feeUp(curveNet, curveConfig.protocolFeeBps);
523     uint256 taxFee = BigShortMath.feeUp(curveNet, uint16(_packedTaxRates[token]));
524     totalCost = curveNet + protocolFee + taxFee;
525     if (totalCost > maxTotalCost) revert SlippageExceeded();
526     if (msg.value < totalCost) revert InsufficientPayment();
527     _applyBuy(
528         token,
529         curve,
530         TradeKind.BuyExactTokens,
531         recipient,
532         tokenAmount,
533         curveNet,
534         protocolFee,
535         taxFee,
536         totalCost,
537         msg.value - totalCost
538     );
539 }
```

Listing 2.6: src/v1/BigShortFactoryV1.sol

```
919 function _applyBuy(
920     address token,
921     Curve storage curve,
922     TradeKind kind,
923     address recipient,
924     uint256 tokenAmount,
925     uint256 curveNet,
926     uint256 protocolFee,
927     uint256 taxFee,
928     uint256 totalCost,
929     uint256 refund
930 ) internal {
931     uint256 virtualTokenBefore = curve.virtualTokenReserves;
932     uint256 virtualNativeBefore = curve.virtualNativeReserves;
```

```
933     uint256 realTokenBefore = curve.realTokenReserves;
934     uint256 realNativeBefore = curve.realNativeReserves;
935     curve.virtualTokenReserves = virtualTokenBefore - tokenAmount;
936     curve.virtualNativeReserves = virtualNativeBefore + curveNet;
937     curve.realTokenReserves = realTokenBefore - tokenAmount;
938     curve.realNativeReserves = realNativeBefore + curveNet;
939     totalRealNativeReserves += curveNet;
940     if (curve.realTokenReserves == 0) curve.phase = Phase.Complete;
941     _collectCurveFees(token, msg.sender, curveNet, protocolFee, taxFee);
942     IERC20(token).safeTransfer(recipient, tokenAmount);
943     if (refund != 0) {
944         (bool ok,) = payable(msg.sender).call{ value: refund }("");
945         if (!ok) revert NativeTransferFailed();
946     }
947     emit CurveTrade(
948         token,
949         msg.sender,
950         recipient,
951         kind,
952         tokenAmount,
953         curveNet,
954         protocolFee,
955         taxFee,
956         uint16(_packedTaxRates[token]),
957         totalCost,
958         lifetimeCurveTaxAssessedNative[token],
959         curve.virtualTokenReserves,
960         curve.virtualNativeReserves,
961         curve.realTokenReserves,
962         curve.realNativeReserves,
963         curve.phase,
964         block.timestamp
965     );
966     if (curve.phase == Phase.Complete) {
967         uint256 migrationTokens = _curveConfigForToken(token).migrationTokenReserves;
968         emit CurveCompleted(token, msg.sender, curve.realNativeReserves, migrationTokens, block
            .timestamp);
969         this.executeAutoMigration(token, msg.sender);
970     }
971     _requireFactorySolvent();
972 }
```

Listing 2.7: src/v1/BigShortFactoryV1.sol

Impact Users may buy base tokens at an unexpected price during the migration.

Suggestion Revise the logic accordingly.

2.1.4 Potential circumvention of liquidations

Severity Medium

Status Fixed in [Version 3](#)

Introduced by [Version 2](#)

Description In the contract `BigShortEngineV1`, the function `_liquidate()` allows whitelisted liquidators to liquidate unhealthy positions. The function `_openOrIncrease()` allows users to increase their positions without applying a health check for the increased position. Moreover, the function `_openOrIncrease()` updates the variable `openedBlockNumber` to block other operations (e.g., position close) in the same block via the check `_requireAfterOpeningBlock()`. However, this design allows malicious users to circumvent liquidations via increasing their positions. Specifically, in the same block, if the targeted position is increased before the liquidation, the liquidation reverts due to the check `_requireAfterOpeningBlock()`. As a result, the malicious users can circumvent liquidations by preemptively increasing their unhealthy positions with the minimum notional value (i.e., the variable `minimumNotionalNative`).

```
217     RiskState memory risk = riskState();
218     if (notionalNative > Math.mulDiv(msg.value, BPS, risk.marginRatioBps)) {
219         revert InsufficientCollateral();
220     }
```

Listing 2.8: `src/v1/BigShortEngineV1.sol`

```
269     position.openedBlockNumber = block.number;
```

Listing 2.9: `src/v1/BigShortEngineV1.sol`

```
429     _requireAfterOpeningBlock(positionId);
```

Listing 2.10: `src/v1/BigShortEngineV1.sol`

```
1015    /// @dev Full/partial closes and single/batch liquidations wait for a block after the latest
1016    open/increase.
1017    function _requireAfterOpeningBlock(uint256 positionId) internal view {
1018        uint256 openedBlockNumber = positions[positionId].openedBlockNumber;
1019        if (block.number <= openedBlockNumber) {
1020            revert PositionBlockLocked(positionId, openedBlockNumber, block.number);
1021        }
```

Listing 2.11: `src/v1/BigShortEngineV1.sol`

Impact Malicious users can circumvent liquidations by preemptively increasing their positions with the minimum notional value.

Suggestion Revise the logic accordingly.

2.1.5 Potential DoS due to the malicious creation of positions

Severity Medium

Status Fixed in [Version 3](#)

Introduced by [Version 1](#)

Description In the contract `BigShortEngineV1`, the function `repayLendingBadDebt()` reverts whenever `totalBorrowedTokens != 0`. However, any user can open a minimal short position and keep it healthy, permanently blocking the repayment operation. As a result, the variable

`unrecoveredLendingTokens` only increases and never decreases, preventing the project from repaying bad debts.

```
1116 function repayLendingBadDebt(uint256 amount) external ready nonReentrant {
1117     _checkpointRiskCapacity();
1118     if (msg.sender != IBigShortEngineFactory(factory).foundation())
1119         revert NotRecoveryAuthority();
1120     if (totalBorrowedTokens != 0) revert OpenBorrowsRemain();
1121     if (amount == 0 || amount > unrecoveredLendingTokens)
1122         revert InvalidAmount();
1123     _requireLendingSolvent();
1124     uint256 beforeBalance = token.balanceOf(address(this));
1125     token.safeTransferFrom(msg.sender, address(this), amount);
1126     uint256 received = token.balanceOf(address(this)) - beforeBalance;
1127     if (received != amount) revert LendingVaultInsolvent();
1128     unrecoveredLendingTokens -= received;
1129     _requireLendingSolvent();
1130     emit LendingBadDebtRepaid(
1131         msg.sender,
1132         received,
1133         unrecoveredLendingTokens
1134     );
1135     _emitEngineStateUpdated();
1136 }
```

Listing 2.12: `src/v1/BigShortEngineV1.sol`

Impact The project cannot repay bad debts due to the malicious creation of positions.

Suggestion Revise the logic accordingly.

2.1.6 Potential sandwich attack targeting liquidations

Severity Medium

Status Fixed in [Version 3](#)

Introduced by [Version 1](#)

Description In the contract `BigShortEngineV1`, the function `_liquidate()` performs the full buyback through the function `_buyExactTokens()` without any slippage bound on the exact-output swap. However, an attacker can perform a sandwich attack by inflating the token price before the liquidations and sell the obtained tokens based on price, which is further inflated by the liquidations. As a result, the project may suffer fund loss due to the sandwich attack targeting liquidations.

```
1017 if (position.marginNative == 0) {
1018     if (amountOutMinimum != 0) revert ShortCloseNotAllowed();
1019 } else if (feasible && required <= position.marginNative) {
1020     report.fullBuyback = true;
1021     report.buybackNative = _buyExactTokens(borrowed, required);
1022     report.recoveredTokens = borrowed;
1023     uint256 remaining = position.marginNative - report.buybackNative;
1024     if (totalShortShares > position.shortShares)
1025         plannedLegacy = remaining / 2;
```

```
1026     plannedGlobal = remaining - plannedLegacy;  
1027 } else {
```

Listing 2.13: src/v1/BigShortEngineV1.sol

Impact The project may suffer fund loss due to the sandwich attack targeting liquidations.

Suggestion Revise the logic accordingly.

2.1.7 Lack of minimum check for the variable `notionalNative` in the function `partialCloseShort()`

Severity Low

Status Confirmed

Introduced by [Version 2](#)

Description In the contract `BigShortEngineV1`, the function `partialCloseShort()` allows users to partially close their short positions. However, this function does not verify whether the remaining notional value (i.e., the variable `notionalNative`) is greater than the minimum requirement (i.e., the variable `minimumNotionalNative`) after the partial closure. As a result, a borrower can partially close their position while leaving its notional value below the variable `minimumNotionalNative`, breaking the intended design.

```
324     uint256 remainingNotional = position.notionalNative - closedNotional;  
325     _recordLegacyReduction(position, remainingNotional);  
326  
327     position.borrowedTokens = borrowedBefore - tokenAmount;  
328     position.shortShares = sharesBefore - closedShares;  
329     position.notionalNative = remainingNotional;
```

Listing 2.14: src/v1/BigShortEngineV1.sol

Impact Users can leave the position notional value below the minimum requirement.

Suggestion Add the minimum check for the position's notional value in the function `partialCloseShort()`.

Feedback from the project The project stated that this behavior is acceptable.

2.2 Recommendation

2.2.1 Revise the approval logic in the function `processExternalTax()`

Status Fixed in [Version 3](#)

Introduced by [Version 1](#)

Description In the contract `BigShortTaxProcessorV1`, the function `processExternalTax()` delegates unlimited approval (i.e., `type(uint256).max`) to the router on each call, which is redundant. It is recommended to revise the approval logic (e.g., add the allowance check) for gas optimization.

```

241     if (!_tryApproveRouter()) {
242         emit RouterApprovalFailed(address(baseToken), baseAmount);
243         _trySplitPayout(_convertWrappedNative(), BigShortTaxPayoutSource.WrappedNativeRetry);
244         return (0, 0, 0);
245     }

```

Listing 2.15: src/v1/BigShortTaxProcessorV1.sol

```

310     function _tryApproveRouter() private returns (bool approved) {
311         (bool ok, bytes memory result) = address(baseToken).call(
312             abi.encodeWithSelector(IERC20.approve.selector, address(router), type(uint256).max)
313         );
314         approved = ok && (result.length == 0 || (result.length == 32 && abi.decode(result, (bool))))
315         );
316     }

```

Listing 2.16: src/v1/BigShortTaxProcessorV1.sol

```

21contract BigShortTokenV1 is ERC20 {

```

Listing 2.17: src/v1/BigShortTokenV1.sol

Suggestion Revise the approval logic in the function `processExternalTax()`.

2.2.2 Unify the native transfer method

Status Fixed in [Version 3](#)

Introduced by [Version 2](#)

Description In the contract `BigShortFactoryV1`, the native transfer is implemented in different ways. It is recommended to unify all native transfers by using the function `_sendNative()`.

```

1147     function _sendNative(address payable recipient, uint256 amount) internal {
1148         (bool ok,) = recipient.call{ value: amount }("");
1149         if (!ok) revert NativeTransferFailed();
1150     }

```

Listing 2.18: src/v1/BigShortFactoryV1.sol

```

662         (bool ok,) = recipient.call{ value: netNativeOut }("");

```

Listing 2.19: src/v1/BigShortFactoryV1.sol

```

946         (bool ok,) = payable(msg.sender).call{ value: refund }("");

```

Listing 2.20: src/v1/BigShortFactoryV1.sol

Suggestion Unify the native transfer method in the contract `BigShortFactoryV1`.

2.2.3 Add a recipient check in the function `claimMigrationDust()`

Status Fixed in [Version 3](#)

Introduced by [Version 2](#)

Description In the contract `BigShortFactoryV1`, the function `claimMigrationDust()` lacks the recipient check `recipient == address(this) || recipient == quoteToken`, which is enforced in the function `claimRejectedCurveFees()`. It is recommended to add the recipient check in the function `claimMigrationDust()`.

```
876 function claimMigrationDust(uint256 amount, address payable recipient) external nonReentrant {
877     if (msg.sender != protocolFeeRecipient) revert NotProtocolFeeRecipient();
878     if (amount == 0 || amount > protocolMigrationDustClaimable || recipient == address(0)) {
879         revert InvalidAmount();
880     }
881     protocolMigrationDustClaimable -= amount;
882     _sendNative(recipient, amount);
883     emit MigrationDustClaimed(recipient, amount, protocolMigrationDustClaimable);
884     _requireFactorySolvent();
885 }
```

Listing 2.21: `src/v1/BigShortFactoryV1.sol`

```
888 function claimRejectedCurveFees(uint256 amount, address payable recipient) external
    nonReentrant {
889     if (msg.sender != protocolFeeRecipient) revert NotProtocolFeeRecipient();
890     if (
891         amount == 0 || amount > rejectedCurveFeesClaimable || recipient == address(0)
892         || recipient == address(this) || recipient == address(quoteToken)
893     ) revert InvalidAmount();
894     rejectedCurveFeesClaimable -= amount;
895     _sendNative(recipient, amount);
896     emit RejectedCurveFeesClaimed(recipient, amount, rejectedCurveFeesClaimable);
897     _requireFactorySolvent();
898 }
```

Listing 2.22: `src/v1/BigShortFactoryV1.sol`

Suggestion Add a recipient check in the function `claimMigrationDust()`.

2.2.4 Remove redundant code

Status Fixed in [Version 3](#)

Introduced by [Version 2](#)

Description There are several redundant code segments in this project. Specifically:

1. In the contract `BigShortFactoryV1`, the function `_migrate()` contains redundant assignment logic.

```
716 if (pair != curve.pair) revert InvalidUniswapConfiguration();
717 registry.validatePair(configurationGroupId, token, pair);
718 curve.pair = pair;
```

Listing 2.23: `src/v1/BigShortFactoryV1.sol`

2. In the contract `BigShortTaxProcessorV1`, the function `_thresholdDirection()` contains the redundant check `expected == 0`.

```

336 function _thresholdDirection(uint256 actualOutput, uint256 expected)
337     private
338     pure
339     returns (int8 direction)
340 {
341     if (actualOutput == 0 || expected == 0) return 0;
342     if (actualOutput > expected) return -1;
343     if (actualOutput < expected) return 1;
344 }

```

Listing 2.24: `src/v1/BigShortTaxProcessorV1.sol`

3. In the contract `BigShortProfileRegistryV1`, the function `_validateDeployer()` contains a duplicated Beacon uniqueness check.

```

121 function _validateDeployer(address deployerAddress) private view {
122     if (deployerAddress.code.length == 0) revert InvalidDeployer();
123     IProtocolDeployer deployer = IProtocolDeployer(deployerAddress);
124     if (deployer.registry() != address(this) || deployer.factory() != factory) revert
        InvalidDeployer();
125     address[4] memory actualBeacons;
126     for (uint256 i; i < 4; ++i) {
127         actualBeacons[i] = deployer.beacon(BusinessKind(i));
128         if (actualBeacons[i] != _beacons[i]
129             || deployer.proxyInitCodeHash(BusinessKind(i)) == bytes32(0)) revert
            InvalidDeployer();
130         for (uint256 j; j < i; ++j) if (actualBeacons[i] == actualBeacons[j]) revert
            InvalidDeployer();
131     }
132 }

```

Listing 2.25: `src/v1/BigShortProfileRegistryV1.sol`

4. In the contract `BigShortFactoryV1`, the error `InvalidQuoteDecimals()` is defined but never used.

```

627     _requireLendingSolvent();

```

Listing 2.26: `src/v1/BigShortEngineV1.sol`

It is recommended to remove redundant code for better readability.

Suggestion Remove redundant code.

2.2.5 Add a sweep mechanism in the contract `BigShortTaxProcessorV1`

Status Fixed in [Version 3](#)

Introduced by [Version 2](#)

Description In the contract `BigShortTaxProcessorV1`, the funds below `taxSwapThreshold` may remain locked if there are no trades. It is recommended to add a sweep mechanism in the contract `BigShortTaxProcessorV1`.

```
249  function processExternalTax()
250      external
251      onlyToken
```

Listing 2.27: src/v1/BigShortTaxProcessorV1.sol

Suggestion Add a sweep mechanism in the contract [BigShortTaxProcessorV1](#).

2.2.6 Unify the reserved storage gap

Status Confirmed

Introduced by [Version 2](#)

Description The project reserves storage slots (i.e., the variable `__gap`) in many implementation contracts for potential upgrades. However, different numbers of slots are reserved in different contracts. It is recommended to unify the reserved gap size across all upgradeable contracts.

```
165  uint256[50] private __gap;
```

Listing 2.28: src/v1/BigShortEngineStorageV1.sol

```
1153  uint256[50] private __gap;
```

Listing 2.29: src/v1/BigShortFactoryV1.sol

```
48  uint256[50] private __gap;
```

Listing 2.30: src/v1/BigShortRiskOracleV1.sol

```
285  uint256[40] private __gap;
```

Listing 2.31: src/v1/BigShortProfileRegistryV1.sol

```
441  uint256[40] private __gap;
```

Listing 2.32: src/v1/BigShortTaxProcessorV1.sol

```
335  uint256[40] private __gap;
```

Listing 2.33: src/v1/BigShortTokenV1.sol

Suggestion Unify the reserved storage gap policy across upgradeable contracts.

2.2.7 Revise the margin-related checks in the contract [EconomicValidation](#)

Status Confirmed

Introduced by [Version 2](#)

Description In the library [EconomicValidation](#), the margin-related checks are improper, which may lead to instant liquidations or bad debts. It is recommended to revise the margin-related checks (i.e., `shortMarginStartBps > shortMarginEndBps` and `shortSafeMarginBps > BPS`).

```
49         || engine.shortRiskEndPriceDropBps >= BPS || engine.shortMarginStartBps < engine.  
           shortMarginEndBps  
50         || engine.shortMarginEndBps < BPS || engine.shortSafeMarginBps < BPS  
51         || engine.fundingStartUtilizationBps >= engine.fundingEndUtilizationBps
```

Listing 2.34: src/v1/libraries/EconomicValidation.sol

```
343     if (position.marginNative < safeMargin) revert InsufficientCollateral();  
344     _requireLendingSolvent();
```

Listing 2.35: src/v1/BigShortEngineV1.sol

```
451     if (feasible && required <= safeBudget) revert PositionHealthy();
```

Listing 2.36: src/v1/BigShortEngineV1.sol

Suggestion Revise the margin-related checks in the contract [EconomicValidation](#).

2.3 Note

2.3.1 Potential centralization risks

Introduced by [Version 2](#)

Description In this project, several privileged roles (e.g., [owner](#)) can conduct sensitive operations, which introduce potential centralization risks. For example, the owner can arbitrarily pause or unpause the entire protocol through the function [setPaused\(\)](#) based on the protocol. If the private keys of the privileged accounts are lost or maliciously exploited, it could pose a significant risk to the protocol.

2.3.2 Ensure correct market grouping

Introduced by [Version 2](#)

Description The contract [BigShortProfileRegistryV1](#) exposes a set of immutable economic parameters via a market group registration mechanism, while the library [EconomicValidation](#) covers the interaction logic for only a subset of these parameters. Since group parameters are locked upon registration, a misconfigured group affects all launched markets. The project must ensure correct market grouping and parameter registration.

2.3.3 Ensure proper adaptations for non-Arc chain deployments

Introduced by [Version 2](#)

Description The project stated that the protocol is designed for the Arc chain, and its code-base relies on Arc-specific features (e.g., the fixed decimal conversion mechanism between native USDC and ERC-20 USDC). The project must ensure that appropriate adaptations (e.g., proper decimal scale) are made before deploying to other chains.

